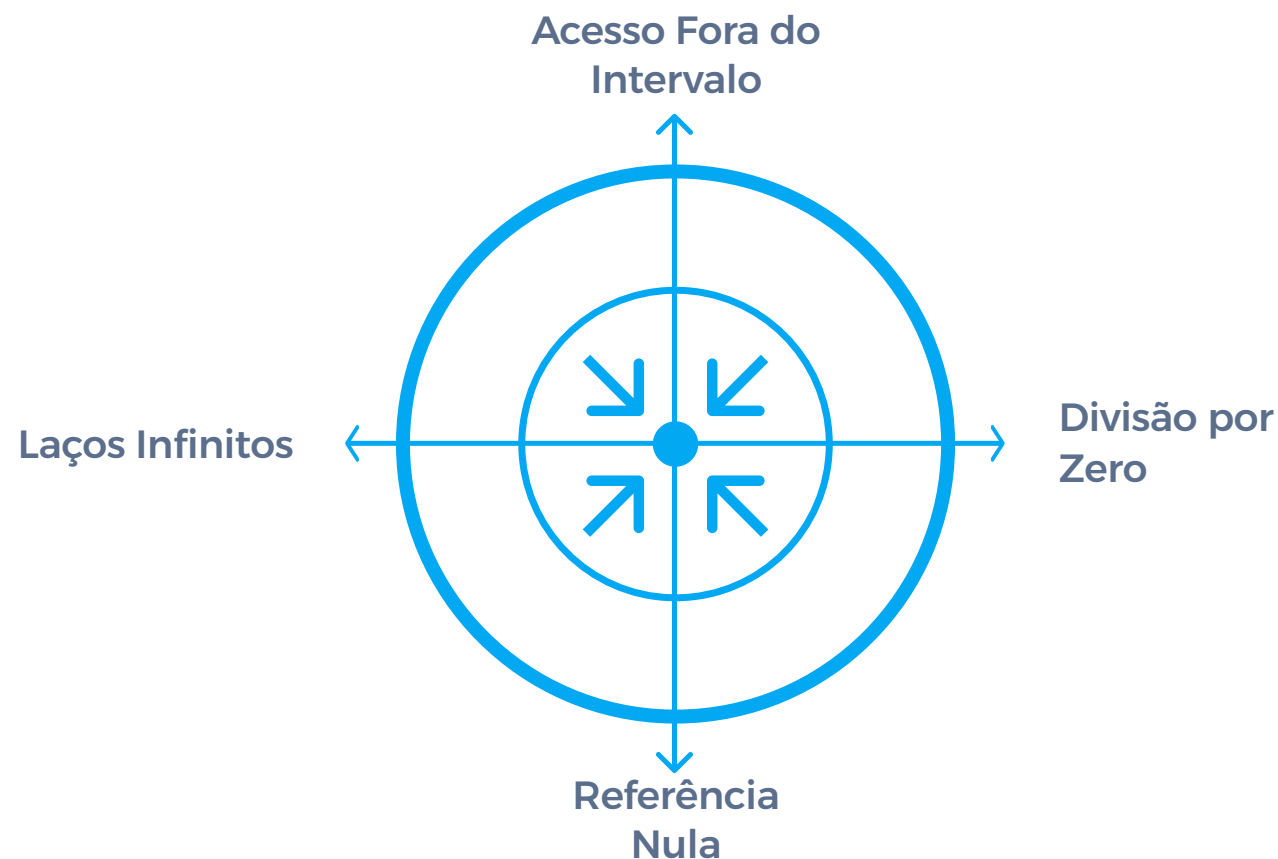


Dafny: Programação com Correção Matemática Garantida

Um guia prático e detalhado sobre a linguagem Dafny — desde sua arquitetura de verificação até a demonstração de teoremas matemáticos complexos, cobrindo especificações, invariantes, lemas e estruturas de dados dinâmicas.



O que é Dafny?

Definição

Dafny é uma linguagem de programação **imperativa, funcional e orientada a objetos** com suporte nativo para **verificação formal estática**.

Ela foi projetada para que desenvolvedores escrevam códigos **matematicamente provados como corretos** em relação às suas especificações — eliminando erros em tempo de compilação.

Erros eliminados em tempo de compilação

Out of Bounds

Acessos a índices fora dos limites de arrays

Divisão por Zero

Operações aritméticas inseguras

Ponteiros Nulos

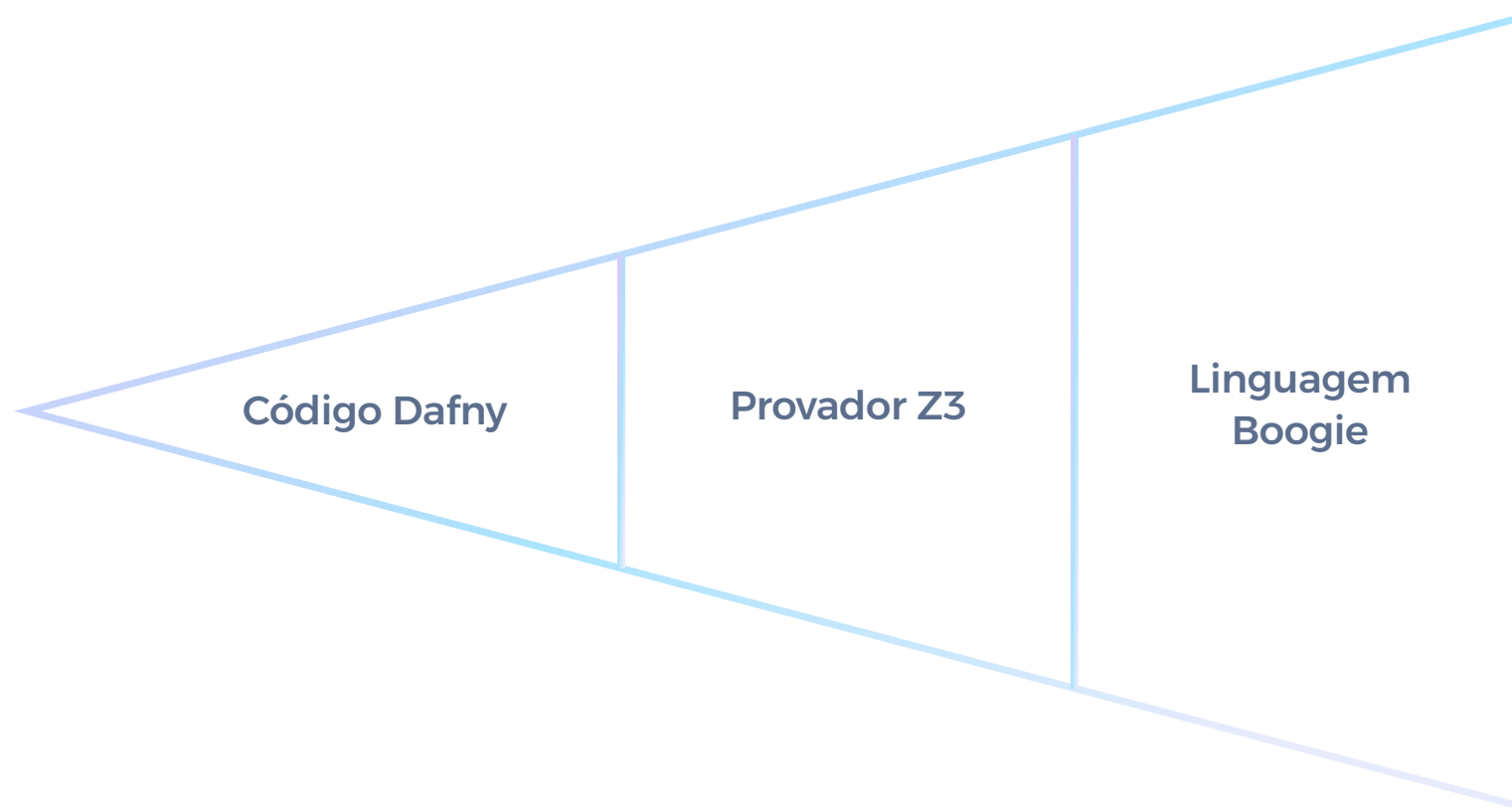
Desreferenciamento de referências nulas

Laços Infinitos

Loops sem garantia de terminação

Arquitetura e Fluxo de Execução

Dafny adota a abordagem de **verificação autoativa (auto-active verification)**, onde o desenvolvedor interage com o provador fornecendo anotações lógicas diretamente no código fonte.



A arquitetura interna de verificação é dividida em três etapas principais, formando um pipeline que vai do código fonte até a certificação matemática de correteude.

As Três Etapas de Verificação

A blue arrow pointing right, containing a code icon (</>).

Tradução para Boogie

O compilador Dafny recebe o código fonte com implementações e especificações e o traduz para **Boogie** — uma linguagem de verificação intermediária (IVL) especializada em semânticas imperativas.

A blue arrow pointing right, containing a VC icon ($f \wedge$).

Geração de VCs

O Boogie processa o programa e gera as **Condições de Verificação (VCs)** — fórmulas matemáticas em lógica de primeira ordem baseadas na técnica de **Precondições Mais Fracas**.

A blue arrow pointing right, containing a checkmark icon (✓).

Análise pelo Z3 (SMT)

As fórmulas são enviadas ao **Z3**, provador de teoremas baseado em SMT. Se nenhuma invalidação for possível, o Dafny declara o programa **semanticamente correto**.

Compilação Após Verificação

O que acontece após a prova?

Após o motor de verificação certificar a corretude do software, o compilador do Dafny traduz o código para uma **linguagem alvo executável**, descartando todas as anotações e elementos puramente lógicos (*ghost*) para garantir desempenho nativo.

- ✓ Todas as anotações lógicas são removidas do binário final — overhead zero em produção.

Linguagens alvo suportadas

C#

Java

JavaScript

Python

Go

Rust

Fundamentos da Especificação: Contratos de Código

A verificação do Dafny baseia-se fortemente na **teoria de contratos**. Para distinguir o que é executável em tempo de execução do que serve exclusivamente para guiar o provador de teoremas, o Dafny define construções normais e construções lógicas (*ghost*).

Palavra-chave	Propósito	Contexto de Uso
<code>requires</code>	Define as pré-condições necessárias antes da execução	Métodos, funções e lemas
<code>ensures</code>	Garante as pós-condições ao término da execução	Métodos, funções e lemas
<code>invariant</code>	Propriedade que se mantém verdadeira em cada iteração	Laços (<code>while</code>)
<code>decreases</code>	Métrica de terminação para garantir ausência de loop infinito	Laços e recursões
<code>modifies</code>	Especifica quais posições de memória do Heap podem ser alteradas	Métodos
<code>reads</code>	Especifica quais posições de memória do Heap podem ser lidas	Funções

Métodos vs. Funções

method

É uma unidade de código **imperativo executável**. Pode alterar estados de memória, ler variáveis e executar loops.

- Permite efeitos colaterais
- Pode usar `modifies`
- Suporta estruturas imperativas
- Parâmetros mutáveis

function

Representa uma **função matemática pura** (sem efeitos colaterais). Implicitamente determinística e executável por padrão nas versões modernas.

- Sem efeitos colaterais
- Usa `reads` para acesso ao Heap
- Parâmetros imutáveis
- Equivalente ao antigo `function method`

 Em versões modernas do Dafny, `function` é executável por padrão — não é mais necessário escrever `function method`.

Valor Absoluto: O Básico de requires e ensures

Este exemplo demonstra como especificar e provar a corretude do cálculo do valor absoluto de um inteiro.

```
method Abs(x: int) returns (y: int)
// Pós-condições: garantem as propriedades matemáticas do resultado
ensures y >= 0
ensures x >= 0 ==> y == x
ensures x < 0 ==> y == -x
{
  if x < 0 {
    return -x;
  } else {
    return x;
  }
}
```

Como o Z3 prova isso?

O provador Z3 analisa os **dois caminhos possíveis** do condicional `if`. Para o caminho `x < 0`, ele substitui `y` por `-x` nas pós-condições e verifica se as fórmulas matemáticas são válidas.

Resultado



A prova é concluída instantaneamente pelo Z3, sem necessidade de anotações adicionais.

Divisão Inteira Segura: Evitando Erros em Tempo de Execução

O Dafny exige que toda operação potencialmente perigosa seja provada segura antes de sua execução. Para realizar uma divisão inteira, devemos garantir que o divisor seja diferente de zero.

```
method SafeDivide(numerator: int, denominator: int)
  returns (quotient: int, remainder: int)
  // Pré-condição: impede divisor igual a zero
  requires denominator > 0
  // Pós-condições: corretude matemática da divisão Euclidiana
  ensures numerator == quotient * denominator + remainder
  ensures 0 <= remainder < denominator
{
  quotient := numerator / denominator;
  remainder := numerator % denominator;
}
```

⚠ Se tentarmos invocar `SafeDivide(10, 0)`, o Dafny **rejeitará a compilação** apontando que a pré-condição `denominator > 0` não foi satisfeita pelo chamador.

Lógica de Floyd-Hoare Aplicada em Dafny

Dafny mapeia diretamente a **Lógica de Floyd-Hoare** para sua estrutura de código. A tripla de Hoare $\{P\}C\{Q\}$ é expressa escrevendo `requires P` antes do corpo do método e `ensures Q` após a assinatura.

1

Inicialização (Base)

O invariante I deve ser verdadeiro imediatamente antes de entrar no laço.

2

Preservação (Indução)

Assumindo que I e a guarda do laço B são verdadeiros, a execução do corpo deve restabelecer I ao fim da iteração.

3

Terminação

Quando o laço termina ($\neg B$), a combinação $I \wedge \neg B$ deve implicar logicamente a pós-condição desejada Q .

Correção Parcial

Garante que se o programa terminar, o resultado será correto.
Mantida pelo uso correto de `invariant`.

Correção Total

Exige, além da correção parcial, a garantia matemática de que o programa *irá terminar*. Realizada pela cláusula `decreases`.

Somatório Simples: Correção Total e Invariantes

Queremos computar o somatório de todos os inteiros de 0 a n . Primeiro, definimos a **função matemática de referência recursiva**:

```
function SumRec(n: nat): nat {  
  if n == 0 then 0 else n + SumRec(n - 1)  
}
```

Agora, escrevemos o método iterativo e usamos o Dafny para provar que a implementação imperativa é matematicamente equivalente à especificação recursiva:

```
method ComputeSum(n: nat) returns (sum: nat)  
  ensures sum == SumRec(n)  
{  
  sum := 0;  
  var i := 0;  
  while i < n  
  invariant 0 <= i <= n  
  invariant sum == SumRec(i) // Correção Parcial  
  decreases n - i // Correção Total  
  {  
    i := i + 1;  
    sum := sum + i;  
  }  
}
```

Análise das Provas do Somatório

1 Inicialização

Antes do laço, $\text{sum} == 0$ e $i == 0$. Como $\text{SumRec}(0) == 0$, o invariante $\text{sum} == \text{SumRec}(i)$ é satisfeito na entrada.

2 Preservação

Com a hipótese de indução $\text{sum} == \text{SumRec}(i)$, após atualizar para $i' = i+1$ e $\text{sum}' = \text{sum} + (i+1)$, o provador verifica:
 $\text{sum}' = \text{SumRec}(i) + (i+1) = \text{SumRec}(i+1)$, restabelecendo o invariante.

3 Terminação

O valor $n - i$ é positivo pela guarda do laço. A cada iteração, i é incrementado em 1, fazendo $n - i$ decrescer estritamente até atingir 0. Como $i \leq n$ é invariante, o laço obrigatoriamente terminará.

Busca Binária Verificada

A busca binária é um algoritmo clássico cuja implementação correta é notoriamente desafiadora. Primeiro, definimos o predicado auxiliar de array ordenado:

```
predicate Sorted(a: array<int>)
  reads a
{
  forall i, j :: 0 <= i < j < a.Length ==> a[i] <= a[j]
}
```

Agora implementamos o método de busca binária com especificação completa:

```
method BinarySearch(a: array<int>, key: int) returns (index: int)
  requires Sorted(a)
  ensures index >= 0 ==> index < a.Length && a[index] == key
  ensures index < 0 ==> forall k :: 0 <= k < a.Length ==> a[k] != key
{
  var low := 0;
  var high := a.Length;
  while low < high
  invariant 0 <= low <= high <= a.Length
  invariant forall i :: 0 <= i < a.Length && !(low <= i < high) ==> a[i] != key
  decreases high - low
  {
    var mid := low + (high - low) / 2;
    if a[mid] < key { low := mid + 1; }
    else if key < a[mid] { high := mid; }
    else { return mid; }
  }
  return -1;
}
```

Como Dafny e Z3 Provam a Busca Binária



Segurança do Acesso ao Array

A divisão $\text{mid} := \text{low} + (\text{high} - \text{low}) / 2$ garante $\text{low} \leq \text{mid} < \text{high}$. Como o invariante assegura $0 \leq \text{low} \leq \text{high} \leq \text{a.Length}$, o Dafny deduz que $0 \leq \text{mid} < \text{a.Length}$. Isso prova estaticamente que $\text{a}[\text{mid}]$ **nunca causará Out of Bounds**.



Preservação Lógica

Se $\text{a}[\text{mid}] < \text{key}$, como o array está ordenado (Sorted), todos os elementos em posições $\leq \text{mid}$ também são estritamente menores que key . Logo, $\text{low} := \text{mid} + 1$ preserva o invariante de que o elemento não reside fora do intervalo ativo.



Terminação Garantida

Como $\text{low} < \text{high}$, a posição mid satisfaz $\text{low} \leq \text{mid} < \text{high}$. Independentemente de alterarmos $\text{low} := \text{mid} + 1$ ou $\text{high} := \text{mid}$, o tamanho do intervalo $\text{high} - \text{low}$ diminui em pelo menos uma unidade.

Lemas e Indução: Dafny como Assistente de Provas

Um dos aspectos mais poderosos do Dafny é sua habilidade de atuar como um **assistente de provas de teoremas matemáticos**. Para provar propriedades complexas, usamos a estrutura `lemma`.



Ghost Method

Um lema é compilado como um **método fantasma**. É executado unicamente em tempo de verificação e completamente removido do binário final — **overhead zero**.



Assinatura = Teorema

Os parâmetros representam as variáveis quantificadas, as cláusulas `requires` definem as premissas, e `ensures` define a **tese a ser demonstrada**.



Corpo = Prova

Escrever o corpo do lema equivale a desenvolver o **passo a passo da prova matemática** — por indução, por casos ou por contradição.

Prova da Fórmula de Gauss para Soma de Naturais

Queremos provar matematicamente o teorema de Gauss, que afirma que o somatório de todos os números naturais de 0 a n pode ser calculado pela fórmula fechada:

$$S(n) = \frac{n(n+1)}{2}$$

```
lemma GaussSumTheorem(n: nat)
  ensures SumRec(n) == n * (n + 1) / 2
{
  if n == 0 {
    // Caso Base:
    // SumRec(0) = 0
    // 0 * (0 + 1) / 2 = 0
    // Dafny aceita trivialmente.
  } else {
    // Passo Indutivo:
    // Hipótese de Indução: fórmula vale para n - 1
    GaussSumTheorem(n - 1);
    // Com a chamada recursiva, o Dafny aprende essa verdade matemática.
    // O provador realiza a expansão algébrica automaticamente:
    // SumRec(n) = n + SumRec(n-1)
    //      = n + (n-1)*n/2
    //      = (2n + n^2 - n)/2
    //      = n*(n+1)/2
  }
}
```

 Em linguagens como Coq ou Lean, o passo algébrico exige aplicação manual de múltiplos teoremas. No Dafny, graças à integração com o Z3, a álgebra não linear simples é resolvida automaticamente assim que as hipóteses indutivas corretas são apresentadas.

Verificação de Estruturas de Dados Dinâmicas: Heap e Arrays

Quando lidamos com algoritmos imperativos que modificam arrays diretamente na memória, precisamos lidar com a noção de **Enquadramento** (Framing).

A cláusula `modifies`

O Dafny utiliza `modifies` para definir o **escopo de modificação de escrita** de um método.

Sem especificar o enquadramento, o provador assume que qualquer objeto na memória Heap pode ter sido corrompido, **inviabilizando a verificação** de propriedades locais.

O operador `old(...)`

Para referenciar o valor que uma variável mutável possuía **antes da chamada do método**, usamos o operador `old(...)`.

❏ Exemplo: `old(a[i])` refere-se ao valor original do elemento `a[i]` antes de qualquer modificação pelo método.

Inversão de Array In-Place com Modificação de Heap

Vamos criar um método para inverter a ordem de todos os elementos de um array de inteiros diretamente na memória (*in-place*) e provar que o estado final corresponde matematicamente à inversão da especificação original.

```
method ReverseArray(a: array<int>)  
  modifies a  
  ensures forall i :: 0 <= i < a.Length ==> a[i] == old(a[a.Length - 1 - i])  
{  
  if a.Length <= 1 { return; }  
  var i := 0;  
  var j := a.Length - 1;  
  while i < j  
  invariant 0 <= i <= a.Length / 2  
  invariant j == a.Length - 1 - i  
  // Elementos externos já processados foram devidamente trocados:  
  invariant forall k :: 0 <= k < i ==>  
    a[k] == old(a[a.Length - 1 - k]) &&  
    a[a.Length - 1 - k] == old(a[k])  
  // Porção interna ainda não alcançada permanece inalterada:  
  invariant forall k :: i <= k <= j ==> a[k] == old(a[k])  
  decreases j - i  
  {  
    a[i], a[j] := a[j], a[i]; // Atribuição simultânea atômica  
    i := i + 1;  
    j := j - 1;  
  }  
}
```

Detalhes Lógicos da Prova de Inversão

1 Permissão de Escrita com `modifies a`

A cláusula `modifies a` permite que o Dafny mude o estado das referências de memória apontadas pelo objeto array `a`. Sem ela, o provador rejeitaria qualquer tentativa de modificação.

2 Invariante de Imutabilidade Parcial

O invariante `forall k :: i <= k <= j ==> a[k] == old(a[k])` é de extrema importância. Ele prova ao Z3 que o loop **não modificou acidentalmente os elementos internos** durante os passos anteriores. Sem essa afirmação, o provador perde o rastro dos valores originais e falha na demonstração da pós-condição.

3 Atribuição Simultânea Atômica

A atribuição `a[i], a[j] := a[j], a[i]` é **atômica no Dafny**, simplificando as asserções da Lógica de Hoare e eliminando o risco de sobreposição de escrita inadvertida – sem necessidade de variável temporária explícita.

Dafny: Síntese do Cookbook

Este cookbook cobriu os principais pilares da programação verificada com Dafny, desde a arquitetura interna até provas matemáticas avançadas.



Com Dafny, a correção do software deixa de ser uma esperança e passa a ser uma **garantia matemática verificada em tempo de compilação** — sem custo em tempo de execução.

6

Linguagens alvo

C#, Java, JS, Python, Go, Rust

3

Etapas de verificação

Dafny → Boogie → Z3

0

Overhead em produção

Anotações ghost removidas do binário